BÀI 7. HỆ THỐNG ĐẢM BẢO CHẤT LƯỢNG PHẦN MỀM

MỤC LỤC

1. HỆ THỐNG VÀ QUY TRÌNH ĐẢM BẢO CHẤT LƯỢNG PHẦN MỀM 2
1.1. Hệ thống đảm bảo chất lượng 2
1.2. Quy trình đảm bảo chất lượng 3
2. MỤC TIÊU VÀ YÊU CẦU CHẤT LƯỢNG 4
2.1. Mục tiêu chất lượng 4
2.2. Yêu cầu chất lượng 5
3. XÁC ĐỊNH MÔ HÌNH PHÁT TRIỂN DỰ ÁN 6
4. THIẾT LẬP CÁC TIÊU CHUẨN, THỦ TỤC VÀ TÀI LIỆU QA 7
4.1. Tiêu chuẩn tài liệu 7
4.2. Thiết lập quy định về coding 9
4.3. Thiết lập quy trình review 9
4.4. Thiết lập quy trình kiểm thử 10
4.5. Thiết lập danh mục kiểm tra 11
4.6. Thiết lập thủ tục QA 12
5. XÁC ĐỊNH DANH MỤC KIỂM TRA 14
6. CÔNG CỤ HỖ TRỢ 17

BÀI 7. HỆ THỐNG ĐẢM BẢO CHẤT LƯỢNG PHẦN MỀM

1. HỆ THỐNG VÀ QUY TRÌNH ĐẢM BẢO CHẤT LƯỢNG PHẦN MỀM

1.1. Hệ thống đảm bảo chất lượng

Hệ thống đảm bảo chất lượng phần mềm bao gồm các thành phần: chính sách, quy trình, tiêu chuẩn, kế hoạch, công cụ và hoạt động được tổ chức nhằm đảm bảo rằng sản phẩm phần mềm đạt yêu cầu về chất lượng trong suốt vòng đời phát triển. Không chỉ đơn thuần là kiểm thử sản phẩm, hệ thống SQA bao gồm ba thành phần chính:

(1) Thiết lập kế hoạch và tiêu chuẩn chất lượng;

(2) Kiểm soát quá trình và sản phẩm thông qua kiểm thử và đánh giá;

(3) Cải tiến liên tục dựa trên phản hồi và đo lường chất lượng.

Việc xây dựng hệ thống SQA đòi hỏi phải thiết lập rõ ràng các thành phần như: mục tiêu và yêu cầu chất lượng, mô hình phát triển phần mềm, tiêu chuẩn tài liệu, quy định về lập trình (coding standards), quy trình kiểm thử và review, cũng như hệ thống các danh mục kiểm tra (checklist). Các công cụ hỗ trợ như SonarQube, TestRail, Jira hay Google Forms cũng là một phần quan trọng trong việc theo dõi, đo lường và cải tiến chất lượng.

Tài liệu hướng dẫn thiết lập hệ thống SQA thường dựa trên các tiêu chuẩn quốc tế như IEEE Std 730-2014 (hướng dẫn xây dựng Kế hoạch SQA), ISO/IEC 12207:2017 (quy trình vòng đời phần mềm), ISO/IEC 25010:2011 (mô hình chất lượng sản phẩm phần mềm), và các khung phương pháp như CMMI for Development v2.0. Việc triển khai đầy đủ các thành phần này giúp tổ chức kiểm soát hiệu quả chất lượng phần mềm, nâng cao tính nhất quán, độ tin cậy và khả năng bảo trì của sản phẩm.

IEEE STD 730-2014 hướng dẫn thiết lập hệ thống SQA gồm 5 thành phần:

(1) Xác định phạm vi và mục tiêu của hệ thống SQA

+ Xác định sản phẩm phần mềm và các thành phần cần được kiểm soát chất lượng.
+ Xác định mục tiêu của hệ thống SQA (ví dụ: đảm bảo độ tin cậy, hiệu năng, bảo mật...).
+ Liên kết với các yêu cầu nghiệp vụ, mục tiêu dự án và tiêu chuẩn ngành (ISO/IEC 25010).

(2) Xây dựng kế hoạch đảm bảo chất lượng phần mềm

(3) Thiết lập tổ chức và nguồn lực SQA
+ Thành lập nhóm SQA: gồm QA leader, testers, auditor (nếu có).
+ Phân vai trách nhiệm rõ ràng: người phụ trách kiểm thử, review, đo lường, báo cáo.

(4) Triển khai các quy trình và tiêu chuẩn QA
+ Xây dựng quy trình kiểm thử, review, coding standard, checklist.
+ Triển khai theo các giai đoạn vòng đời phần mềm (tùy mô hình: Waterfall, Agile…).

(5) Thực hiện đo lường, đánh giá và cải tiến
+ Thu thập số liệu: defect density, % test case pass, time to resolve bugs, coverage…
+ Đánh giá định kỳ: audit nội bộ, họp retrospective, điều chỉnh quy trình nếu cần.

1.2. Quy trình đảm bảo chất lượng

Quy trình đảm bảo chất lượng phần mềm là một hệ thống các hoạt động được lên kế hoạch và có tổ chức nhằm đảm bảo rằng các quy trình, thủ tục và tiêu chuẩn được sử dụng trong quá trình phát triển phần mềm đáp ứng các yêu cầu chất lượng đã đề ra. Quy trình đảm bảo chất lượng phần mềm bao gồm kiểm soát chất lượng (QC), đảm bảo chất lượng (QA) và đánh giá chất lượng (QE):

- Đảm bảo chất lượng: Thiết lập các chính sách, tiêu chuẩn và quy trình để định hướng chất lượng trong toàn bộ vòng đời phát triển phần mềm.

- Kiểm soát chất lượng: Thực hiện các hoạt động cụ thể như kiểm thử, rà soát mã nguồn để phát hiện và loại bỏ lỗi.

- Đánh giá chất lượng: Đánh giá kết quả kiểm tra và quá trình phát triển nhằm đề xuất cải tiến.

Nhóm SQA thường thực hiện quy trình chất lượng theo ba giai đoạn chính:

(1) Lập kế hoạch đảm bảo chất lượng (giai đoạn chuẩn bị);

(2) Kiểm soát chất lượng (triển khai);

(3) Đánh giá và cải tiến chất lượng.

2. MỤC TIÊU VÀ YÊU CẦU CHẤT LƯỢNG

Mục tiêu và yêu cầu chất lượng đóng vai trò nền tảng trong quá trình phát triển phần mềm, đặc biệt khi áp dụng hệ thống QA. Các tài liệu giúp xác định mục tiêu và yêu cầu chất lượng: (1) Yêu cầu người dùng, (2) phạm vi dự án.

2.1. Mục tiêu chất lượng

Mục tiêu chất lượng (Quality goals) định hướng tổng thể cho dự án, giúp xác định "phần mềm cần đạt được những gì?" về mặt chất lượng. Mục tiêu chất lượng là những định hướng quan trọng trong dự án phát triển phần mềm, nhằm đảm bảo sản phẩm cuối cùng đáp ứng được các yêu cầu về tính năng, hiệu năng, độ tin cậy, bảo mật và khả năng bảo trì. Mục tiêu chất lượng không chỉ giúp đội ngũ phát triển xác định rõ tiêu chuẩn cần đạt mà còn là cơ sở để lập kế hoạch kiểm thử, đánh giá và cải tiến trong suốt vòng đời phần mềm.

Trong một dự án phần mềm, các mục tiêu chất lượng thường xuất phát từ:

- Nhu cầu của khách hàng;
- Các tiêu chuẩn ngành (như ISO/IEC 25010, TCVN)
- Yêu cầu nghiệp vụ.

Việc xác định mục tiêu chất lượng ngay từ đầu dự án giúp các bên liên quan có cùng một hiểu biết chung về kỳ vọng đối với phần mềm. Các mục tiêu này cần đo lường được, khả thi và phù hợp với phạm vi dự án. Khi có mục tiêu rõ ràng, nhóm phát triển sẽ lựa chọn được công cụ, kỹ thuật và quy trình phù hợp để hiện thực hóa, đồng thời dễ dàng theo dõi tiến độ và đánh giá mức độ hoàn thành.

2.2. Yêu cầu chất lượng

Yêu cầu chất lượng (Quality requirements) là các tiêu chí cụ thể, đo lường được để đạt mục tiêu. Chúng trả lời câu hỏi "Làm thế nào để biết phần mềm đạt chất lượng?". Yêu cầu chất lượng có thể được thiết lập dựa trên 1 số tiêu chí thuộc tiêu chuẩn ngành (tiêu chuẩn ngành được xác định khi xác định mục tiêu của dự án).

Yêu cầu chất lượng thường đề cập và nhằm xác định các đặc tính cần có của phần mềm để đảm bảo sản phẩm không chỉ hoạt động đúng mà còn hiệu quả, an toàn, dễ sử dụng và dễ bảo trì. Không giống như các yêu cầu chức năng – mô tả phần mềm làm gì – yêu cầu chất lượng trả lời cho câu hỏi “làm như thế nào” để phần mềm thực hiện hành vi đó trong thực tế triển khai.

Theo tiêu chuẩn ISO/IEC 25010:2011, các yêu cầu chất lượng được phân loại thành các nhóm như:

- Tính khả dụng (usability)
- Độ tin cậy (reliability)
- Hiệu năng (performance efficiency)
- Khả năng bảo trì (maintainability)
- Khả năng tương thích (compatibility)
- An toàn thông tin (security)
- Khả năng chuyển giao (portability)
- Tính đúng đắn về mặt chức năng (functional suitability).

Việc xác định rõ yêu cầu chất lượng ngay từ đầu giúp định hình thiết kế kiến trúc phần mềm, lựa chọn công nghệ phù hợp, thiết lập các tiêu chí kiểm thử và giám sát chất lượng trong quá trình phát triển. Nếu bỏ qua hoặc xác định không rõ ràng, phần mềm có thể vẫn đúng về chức năng nhưng lại kém hiệu quả, dễ lỗi hoặc không được người dùng chấp nhận.

3. XÁC ĐỊNH MÔ HÌNH PHÁT TRIỂN

Trong mỗi dự án phần mềm, việc lựa chọn và thiết lập mô hình phát triển phần mềm đóng vai trò then chốt, ảnh hưởng trực tiếp đến hiệu quả quản lý tiến độ, chất lượng sản phẩm và chi phí phát triển. Mô hình phát triển phần mềm chính là khung định hướng quy trình thực hiện các hoạt động từ phân tích yêu cầu, thiết kế, lập trình, kiểm thử cho tới bảo trì.

Các mô hình phổ biến hiện nay bao gồm: Mô hình thác nước (Waterfall), mô hình lặp và tăng trưởng (Iterative/Incremental), mô hình xoắn ốc (Spiral), và các mô hình dựa trên phương pháp linh hoạt (Agile) như Scrum. Mỗi mô hình có cách tổ chức công việc khác nhau, từ tuần tự chặt chẽ đến linh hoạt thích ứng.

Mô hình phát triển ảnh hưởng lớn đối với hoạt động SQA, mô hình phát triển được thiết lập sẽ định hình cách tổ chức các biện pháp kiểm soát chất lượng phù hợp. Chẳng hạn, với mô hình Waterfall, các hoạt động SQA cần được thực hiện rõ ràng ở từng giai đoạn và có kiểm soát chuyển giao pha. Trong khi đó, với mô hình Agile, các hoạt động đảm bảo chất lượng lại phải linh hoạt, tích hợp liên tục với các vòng kiểm thử và đánh giá ngắn hạn. Việc xác định mô hình phát triển từ sớm giúp nhóm QA lựa chọn được chiến lược kiểm thử phù hợp, xác định điểm kiểm soát chất lượng, tài liệu cần thiết và cách thức phối hợp với nhóm phát triển. Ngoài ra, mô hình phát triển còn ảnh hưởng đến việc lập kế hoạch chất lượng, phân bổ tài nguyên QA và quy trình báo cáo lỗi. Do đó, thiết lập mô hình phát triển phần mềm là tiền đề quan trọng để đảm bảo chất lượng được quản lý hiệu quả và nhất quán trong suốt vòng đời dự án.

Một số tiêu lựa chọn mô hình phát triển phần mềm:

- Phù hợp với đặc thù dự án
- Lựa chọn dựa trên mức độ rõ ràng của yêu cầu ban đầu
- Lựa chọn dựa trên độ lớn và độ phức tạp của dự án
- Lựa chọn dựa trên thời gian thực hiện và ngân sách cho dự án
- Lựa chọn dựa trên khả năng tương tác với khách hàng
- Lựa chọn dựa trên mức độ kiểm soát và tiêu chuẩn hóa quy trình

4. THIẾT LẬP CÁC TIÊU CHUẨN, THỦ TỤC VÀ TÀI LIỆU QA

4.1. Tiêu chuẩn tài liệu

Thiết lập các tiêu chuẩn tài liệu có nhiệm vụ chuẩn bị các tài liệu liên quan (tài liệu chuẩn). Danh mục các tài liệu đảm bảo chất lượng phần mềm (SQA documentation) gồm:

(1) Kế hoạch đảm bảo chất lượng: Xác định mục tiêu và yêu cầu chất lượng của dự án, bao gồm cả tiêu chuẩn chất lượng áp dụng, quy trình kiểm thử, checklist kiểm tra code, và kế hoạch review trong các giai đoạn phát triển phần mềm. Tài liệu: QA_Plan.docx.

(2) Quy trình phát triển: Bao gồm các nhiệm vụ và phân vai cho từng thành viên. Mô tả quy trình làm việc của nhóm phát triển theo mô hình quy trình phần mềm đã chọn. Tài liệu: Dev_Workflow_Design.docx.

(3) Chuẩn bố cục và trình bày các tài liệu: Quy định mẫu và dạng chung cho toàn bộ tài liệu của dự án: bố cục, quy định đánh số mục và tiểu tục, quy tắc đặt tên tài liệu, Font (name, size), ….. Tài liệu: Doc_Stand_Guide.docx

(4) Tài liệu đặc tả yêu cầu phần mềm. Đặc tả các yêu cầu chức năng và phi chức năng của hệ thống, làm cơ sở để phát triển, kiểm thử và đánh giá phần mềm. Tài liệu: SRS.docx (Software Requirements Specification)

**** Lưu ý:*** mỗi tính năng/chức năng nên được đặc tả rõ ràng về mặt nghiệp vụ trong tài liệu độc lập (ví dụ: **Login**_Spec.docx, **ManageStudent**_Spec.docx). Tài liệu

này tập trung vào các khía cạnh: hành vi mong đợi, quy tắc nghiệp vụ, dữ liệu đầu vào/ra, và điều kiện tiên quyết. Các tài liệu đặc tả được lưu trong thư mục Function_Specs, liên kết chặt chẽ với SRS.docx, giúp nhóm phát triển và kiểm thử hiểu đúng mong đợi của người dùng. Quy tắc đặt tên: **Chứcnăng_** Specs.docx.

(5) Tài liệu thiết kế phần mềm: Mô tả kiến trúc phần mềm, thiết kế hệ thống và các module thành phần. Nội dung bao gồm sơ đồ kiến trúc, cơ sở dữ liệu, luồng xử lý chính. Tài liệu: SDD.docx (Software Design Document)

**** Lưu ý***: Mỗi tính năng/chức năng lớn của hệ thống có thể có một tài liệu phân tích – thiết kế riêng (ví dụ: **Login_**Design.docx, **SearchStudent_**Design.docx). Các tài liệu này làm rõ yêu cầu cụ thể, luồng xử lý, các lớp đối tượng liên quan, mockup giao diện (nếu có), và tương tác với các module khác. Dạng file nên được lưu trong thư mục Function_Designs và được quản lý phiên bản. Quy tắc đặt tên: **Chứcnăng_**Design.docx.

(6) Kế hoạch kiểm thử: Mô tả chiến lược kiểm thử tổng thể: phạm vi, phương pháp, công cụ, nguồn lực, lịch trình và loại kiểm thử (unit test, integration test, system test). Tài liệu: Test_Plan.docx

(7) Danh sách test case**:** Bao gồm danh sách các trường hợp kiểm thử cụ thể được viết dựa trên yêu cầu phần mềm. Mỗi test case gồm ID, bước thực hiện, đầu vào, kết quả mong đợi và trạng thái. Tài liệu: Test_Cases.docx

(8) Checklist review mã nguồn: bao gồm tiêu chí dùng để kiểm tra mã nguồn. Tài liệu: Code_Review_Checklist.docx

(9) Báo cáo lỗi: Ghi lại chi tiết các lỗi được phát hiện trong quá trình kiểm thử: mô tả lỗi, mức độ nghiêm trọng, người phát hiện, người xử lý, trạng thái xử lý. Tài liệu: Defect_Report.docx.

(10) Báo cáo tổng kết kiểm thử: Ghi lại số lượng test case pass/fail, thống kê lỗi, tổng hợp kết quả kiểm thử. Tài liệu: Test_Summary_Report.docx.

(11) Báo cáo retrospective (nếu có): Tài liệu tổng kết sau mỗi sprint: những gì đã làm tốt, chưa tốt và bài học kinh nghiệm cho các sprint tiếp theo (nếu có). Tài liệu: Retrospective.Docx.

4.2. Thiết lập quy định về coding

Việc thiết lập quy định về coding (coding standards) là một phần thiết yếu trong đảm bảo chất lượng phần mềm, nằm trong hoạt động QA – kiểm soát quá trình phát triển để đảm bảo đầu ra đạt chất lượng.

Thiết lập hướng dẫn quy tắc đặt tên, comment, quy ước cấu trúc file, format code, quy định refactor... Thiết lập quy định về code giúp tăng tính nhất quán (consistency), hỗ trợ code review, hỗ trợ kiểm thử và tự động hóa (CI/CD).

Các quy định coding:

(1) Quy tắc chung:

- Ngôn ngữ lập trình
- Phiên bản
- IDE/Editor

(2) Quy tắc đặt tên:

- Tên biến (Variables)
- Tên hàm (Functions)
- Tên lớp (Classes)
- Hằng số (Constants)

(3) Quy tắc mã nguồn:

- Cấu trúc code
- Tài liệu tham khảo quy tắc mã nguồn (nếu có).

4.3. Thiết lập quy trình review

Xác định rõ trình tự thực hiện hoạt động review trong vòng đời phát triển phần mềm, đảm bảo chất lượng và phát hiện lỗi sớm.

Quy trình review được áp dụng cho hợp đồng, tài liệu yêu cầu (SRS), thiết kế (SDD), mã nguồn. Các bước thực hiện:

(1) Chuẩn bị: chọn đối tượng review (SRS, thiết kế, code...), phân công người review

(2) Phân phối tài liệu: gửi tài liệu cho các reviewer cùng với checklist kiểm tra

(3) Thực hiện review cá nhân: mỗi reviewer rà soát và ghi chú phát hiện

(4) Họp tổng hợp (optional): thảo luận và thống nhất lỗi/cải tiến

(5) Ghi nhận kết quả: lập biên bản review và phân loại mức độ ảnh hưởng

(6) Theo dõi khắc phục: tác giả cập nhật, reviewer xác nhận sửa lỗi

Lưu ý:

- Áp dụng peer review cho tài liệu, và code review với >30% mã thay đổi.
- Reviewer phải độc lập với người viết tài liệu hoặc người viết mã.

4.4. Thiết lập quy trình kiểm thử

Quy trình kiểm thử xác định rõ trình tự thực hiện hoạt động kiểm thử trong vòng đời phát triển phần mềm, đảm bảo chất lượng và phát hiện lỗi sớm. Quy trình kiểm thử phần mềm được áp dụng cho các chức năng phần mềm ở tất cả các cấp độ kiểm thử.

Các bước thực hiện:

(1) Xác định phạm vi kiểm thử: dựa trên SRS, kế hoạch test

(2) Thiết kế test case & test data: tạo test case, dữ liệu tương ứng

(3) Thiết lập môi trường kiểm thử: cấu hình hệ thống, dữ liệu demo

(4) Thực thi kiểm thử: chạy test case và ghi lại kết quả

(5) Ghi nhận lỗi: báo lỗi (nếu có), phân loại theo mức độ nghiêm trọng

(6) Kiểm thử hồi quy (re-test): sau khi lỗi được sửa

(7) Tổng hợp và báo cáo: lập báo cáo kiểm thử (Test Report)

Các cấp độ kiểm thử được áp dụng:

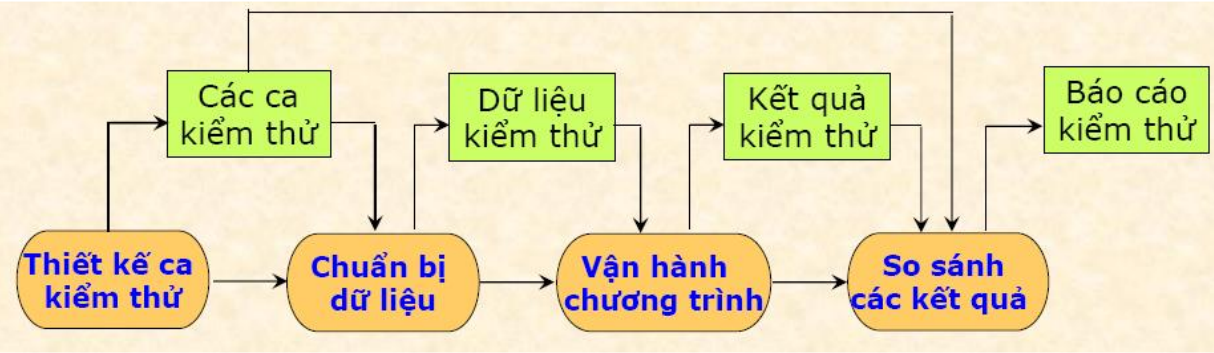
- Unit test: do developer thực hiện.

- Integration test: kiểm tra sự kết hợp giữa các module.

- System test: kiểm tra toàn bộ hệ thống theo luồng nghiệp vụ.

- Acceptance test (UAT): tester kiểm thử (kiểm thử alpha), và sau đó khách hàng xác nhận tính đúng đắn trước khi triển khai (kiểm thử beta).

Tiến trình kiểm thử được minh họa trên Hình 1.



Hình 1. Tiến trình kiểm thử

4.5. Thiết lập danh mục kiểm tra

Danh mục kiểm tra (checklist) là công cụ chuẩn hóa và hệ thống hóa các tiêu chí đánh giá chất lượng, giúp đội ngũ phát triển và SQA tránh bỏ sót lỗi quan trọng. Checklist giúp:

- Giảm sai sót: Tránh bỏ qua lỗi nghiêm trọng do thiếu hệ thống.

- Tiết kiệm thời gian: Không cần "nghĩ lại từ đầu" mỗi lần test/review.

- Nhất quán chất lượng: Áp dụng cùng tiêu chuẩn cho mọi thành viên.

- Đo lường được: Dễ dàng đánh giá tỉ lệ hoàn thành công việc.

Test checklist, code review và Checklist cho tài liệu là các checklist quan trọng trong QA:

- Test checklist: Đảm bảo kiểm thử bao phủ đầy đủ các yêu cầu, chức năng và kịch bản quan trọng.

- Code review: Đảm bảo mã nguồn đáp ứng tiêu chuẩn kỹ thuật, dễ bảo trì và ít lỗi tiềm ẩn.

- Checklist cho tài liệu: dùng cho reviewer chính, để rà soát lại tài liệu xem có đủ các thành phần không? Có bị thiếu các mô tả không.

4.6. Thiết lập thủ tục QA

Thiết lập quy trình/thủ tục trong đảm bảo chất lượng phần mềm (SQA) là việc xây dựng các hướng dẫn chi tiết, bước thực hiện cụ thể và quy tắc chuẩn hóa nhằm kiểm soát chất lượng sản phẩm trong suốt vòng đời phát triển phần mềm thông qua:

- Review (rà soát): Kiểm tra tĩnh (static testing) các tài liệu, mã nguồn, thiết kế.
- Kiểm thử (testing): Kiểm tra động (dynamic testing) để phát hiện lỗi khi chạy hệ thống.

Mục tiêu chính của thiết lập quy trình QA bao gồm:

- Đảm bảo tính nhất quán trong cách làm việc của team.
- Phòng ngừa rủi ro bằng cách tuân thủ các bước đã được kiểm chứng.
- Đo lường được hiệu quả chất lượng thông qua các tiêu chí rõ ràng.

Các quy trình QA cần xây dựng trong bất kỳ dự án phần mềm nào:

(1) Thủ tục kiểm tra mã nguồn:

+ Các mã nguồn phải tuân thủ các quy định về coding standards đã thiết lập (đã đề cập ở trên).

+ Mã nguồn cần phải trải qua quy trình review mã nguồn trước khi được tích hợp vào kho mã chính.

+ Mã cần phải được kiểm tra bằng các công cụ tự động hóa kiểm thử (như SonarQube) để phát hiện các lỗi tiềm ẩn (coding flaws, security vulnerabilities).

(2) Quy trình kiểm thử phần mềm:

+ Trước khi bắt đầu kiểm thử, cần phải có một ***Kế hoạch kiểm thử*** đầy đủ, chi tiết, xác định rõ phạm vi và mục tiêu kiểm thử.

+ Kiểm thử cần phải được thực hiện theo các giai đoạn đã định (unit test, integration test, system test).

+ Tất cả các trường hợp kiểm thử phải được ghi lại trong danh sách Test Cases, với kết quả kiểm thử chi tiết và báo cáo các lỗi phát hiện.

(3) Quy trình báo cáo lỗi:

+ Lỗi phát hiện trong quá trình kiểm thử cần được ghi lại đầy đủ trong Defect Report, bao gồm mức độ nghiêm trọng, mô tả lỗi, trạng thái xử lý và lịch trình xử lý.

+ Các lỗi quan trọng hoặc nghiêm trọng phải được giải quyết ngay lập tức, trong khi các lỗi ít nghiêm trọng có thể được xử lý trong các sprint sau.

(4) Quy trình kiểm tra hiệu suất:

+ Kiểm thử hiệu suất (performance testing) cần được thực hiện để đảm bảo phần mềm có thể đáp ứng các yêu cầu về hiệu suất đã được xác định trong tài liệu yêu cầu.

+ Các công cụ kiểm thử hiệu suất như JMeter có thể được sử dụng để kiểm tra khả năng chịu tải và các vấn đề tiềm ẩn trong khi sử dụng phần mềm với lượng người dùng lớn.

(5) Quy trình bảo trì chất lượng phần mềm:

+ Sau khi phần mềm đã được phát triển và triển khai, vẫn cần có các thủ tục để duy trì chất lượng phần mềm thông qua các quy trình kiểm thử định kỳ và cập nhật.

+ Tất cả các lỗi được phát hiện sau khi triển khai cần được ghi nhận và xử lý kịp thời.

5. XÁC ĐỊNH DANH MỤC KIỂM TRA

Danh mục kiểm tra (checklist) là công cụ chuẩn hóa và hệ thống hóa các tiêu chí đánh giá chất lượng, giúp đội ngũ phát triển và SQA tránh bỏ sót lỗi quan trọng. Checklist giúp:

- Giảm sai sót: Tránh bỏ qua lỗi nghiêm trọng do thiếu hệ thống.
- Tiết kiệm thời gian: Không cần "nghĩ lại từ đầu" mỗi lần test/review.
- Nhất quán chất lượng: Áp dụng cùng tiêu chuẩn cho mọi thành viên.
- Đo lường được: Dễ dàng đánh giá tỉ lệ hoàn thành công việc.

Test checklist, code review và Checklist cho tài liệu là các checklist quan trọng trong QA:

- Test checklist: Đảm bảo kiểm thử bao phủ đầy đủ các yêu cầu, chức năng và kịch bản quan trọng.
- Code review: Đảm bảo mã nguồn đáp ứng tiêu chuẩn kỹ thuật, dễ bảo trì và ít lỗi tiềm ẩn.
- Checklist cho tài liệu: dùng cho reviewer chính, để rà soát lại tài liệu xem có đủ các thành phần không? Có bị thiếu các mô tả không.

Bảng 1. Danh sách các checklist thường dùng trong dự án phát triển phần mềm

🔍 Dự án: Xây dựng và áp dụng hệ thống đảm bảo chất lượng cho dự án phần mềm quản lý sinh viên Trường Đại học Công nghiệp Hà Nội:

1. Checklist review tài liệu yêu cầu (SRS)

TT	Nội dung kiểm tra	Tiêu chí đạt	Kết quả (✓/✗)	Ghi chú
1	Đầy đủ chức năng nghiệp vụ	Có mục: Đăng ký học phần, Xem điểm, Tính điểm TB, Quản lý SV		
2	Yêu cầu phi chức năng	Hiệu năng: ≥1000 SV truy cập đồng thời		
3	Thuật ngữ thống nhất	Dùng "Học phần" thay vì "Môn học"		

2. Checklist thiết kế DATABASE

TT	Bảng cần kiểm tra	Trường bắt buộc	Ràng buộc	Kết quả	Ghi chú

1	`SinhVien`	`MaSV` (PK), `HoTen`, `Lop`	`MaSV` là số 8 ký tự		
2	`Diem`	`MaSV` (FK), `MaHP`, `DiemThi`	`DiemThi` từ 0 đến 10		

3. Checklist coding standard (BACKEND - JAVA)

TT	Hạng mục	Quy tắc	Ví dụ đạt chuẩn	Kết quả
1	Đặt tên biến	camelCase, không dùng tiếng Việt	studentCode thay vì maSV	
2	Xử lý ngoại lệ	Bắt SQLException khi truy vấn database	try-catch rõ ràng	

4. Checklist test case chức năng "đăng ký học phần"

TT	Test case	Đầu vào	Kết quả mong đợi	Thực tế
1	Đăng ký thành công	SV chọn 3 học phần hợp lệ	Hiển thị thông báo "Thành công"	
2	Đăng ký vượt tín chỉ tối đa	SV chọn 6 học phần	Báo lỗi "Vượt quá 5 tín chỉ"	

5. Checklist UI/UX giao diện sinh viên

TT	Thành phần	Tiêu chí	Ảnh minh họa	Đạt/chưa
1	Trang đăng nhập	Có nút "quên mật khẩu"	[link]	
2	Trang xem điểm	Hiển thị dạng bảng có sắp xếp theo học phần	[link]	

6. Checklist bảo mật (áp dụng cho QE)

TT	Hạng mục	Phương pháp kiểm tra	Công cụ	Kết quả
1	SQL INJECTION	Nhập `' or '1'='1` vào ô tìm kiếm sv	Burp suite	
2	XSS	Nhập `<script>alert(1)</script>`	Owasp zap	

7. Checklist triển khai

TT	Công việc	Yêu cầu	Người phụ trách	Hoàn thành
1	Backup database	Sao lưu toàn bộ dữ liệu production	Nguyễn Văn A	
2	Kiểm tra server	CPU ≤ 80%, ram ≥ 4Gb	Phạm Thị D	

8. Checklist đánh giá chất lượng cuối dự án

TT	Chỉ số	Mục tiêu	Thực tế	Đánh giá
1	Tỷ lệ test case pass	≥ 95%		
2	Số lỗi critical	0		

9. QA test checklist

TT	Test checklist	Đã kiểm tra	Ghi chú
1	User story có tiêu chí chấp nhận rõ ràng (Acceptance criteria)		
2	Có unit test cho tất cả các hàm nghiệp vụ chính		
3	Tỷ lệ bao phủ kiểm thử (Test coverage) đạt ≥ 80%		
4	Chạy thành công các kiểm thử tự động (CI/CD)		
5	Không có bug nghiêm trọng (Severity 1 hoặc 2) tồn tại sau Sprint		
6	Kết quả kiểm thử được ghi nhận trong hệ thống quản lý test (nếu có)		
7	Kiểm thử hồi quy được thực hiện sau mỗi lần cập nhật code chính		
8	Đáp ứng yêu cầu hiệu năng: < 2s cho truy vấn điểm		
9	Trình bày UI/UX đúng bản thiết kế		

10. Code review checklist

TT	Tiêu chí	Mức độ ưu tiên	Ghi chú
1	Code dễ đọc, có đặt tên biến/hàm rõ ràng		
2	Không lặp mã (code duplication)		
3	Comment rõ ràng nếu đoạn code phức tạp		
4	Tuân thủ quy ước code chuẩn (coding convention)		
5	Không commit mã test/thử nghiệm vào nhánh chính		
6	Xử lý ngoại lệ rõ ràng (error handling)		
7	Có test tương ứng nếu thêm chức năng mới		
8	Không để lại mã thừa, mã bị comment bỏ		

6. CÔNG CỤ HỖ TRỢ

Theo dõi chất lượng mã nguồn: SonarQube (kiểm tra chất lượng code: lỗi coding style, bugs tiềm ẩn,...), CodeClimate,…

Quản lý tài liệu và cải tiến quy trình: Confluence (dùng ghi nhận Retrospective và kế hoạch cải tiến), Notion,…

Thu thập phản hồi người dùng: Google Forms (tạo khảo sát nhanh với người dùng sau khi UAT), UserVoice,…

Quản lý kiểm thử: TestRail, Zephyr;

Theo rõi lỗi: Jira, Bugzilla;

Tự động hóa kiểm thử: Selenium, Postman